

Evaluación Unidad 2

Resultado de Aprendizaje:

1.2 Programa algoritmos para el procesamiento y transformación de datos.

Indicadores de Logro:

- Diseña algoritmos utilizando estructuras de control y funciones que resuelven problemas básicos de procesamiento y transformación de datos.
- Implementa en un lenguaje de programación (ej. Python, R, Java) algoritmos que realicen operaciones como limpieza, normalización, agregación y combinación de datos.
- Aplica técnicas de programación estructurada y modular para desarrollar algoritmos eficientes, garantizando claridad, legibilidad y reutilización del código.
- Valida algoritmos en distintos conjuntos de datos, de acuerdo a pruebas, evaluando su eficacia en la transformación de datos estructurados y no estructurados para su posterior análisis.

Formato de entrega.

- Debe incluir portada, la que debe ser del IP San Sebastián.
- Interlineado 1.15, letra Arial 12, títulos 14.
- El documento debe estar con su texto justificado.
- Trabajo individual
- Códigos fuentes, Pseudocódigo, diagrama de flujo del pipeline, dataset finales y consolidados y registros de validación. También debe entregar README con instrucciones de ejecución, Test básicos. Supuestos.
- Documento Word con lo que se solicita más abajo en la explicación.
- Bibliografía en formato APA 7

Evaluación Unidad 2

CASO DE ESTUDIO: ANÁLISIS INTEGRADO DEL SISTEMA EDUCACIONAL Y FACTORES SOCIOECONÓMICOS EN CHILE

1. ANTECEDENTES Y CONTEXTO

El Ministerio de Educación de Chile, en colaboración con el Instituto Nacional de Estadísticas (INE), ha iniciado un proyecto para comprender mejor la relación entre el rendimiento educacional y diversos factores socioeconómicos a nivel regional. Este análisis es fundamental para la toma de decisiones en políticas públicas educativas y la asignación eficiente de recursos.

Actualmente, la información está dispersa en múltiples fuentes de datos que presentan diversos problemas de calidad: valores faltantes en registros históricos, inconsistencias en formatos de fechas y nombres de regiones, duplicados por errores de registro, y valores atípicos que requieren validación. Además, los datos provienen de diferentes sistemas (bases de datos gubernamentales, encuestas, registros administrativos) y están en distintos formatos (CSV, JSON).

La Subsecretaría de Educación requiere un análisis consolidado que integre:

- Datos de rendimiento escolar por establecimiento y región
- Indicadores socioeconómicos regionales (ingreso promedio, población, índice de vulnerabilidad)
- Información de infraestructura educativa (número de establecimientos, matrícula, recursos)

Esta integración permitirá identificar brechas educativas, priorizar intervenciones y evaluar el impacto de factores contextuales en los resultados de aprendizaje.

2. PROBLEMA CENTRAL

Los datos educacionales y socioeconómicos de Chile están dispersos, desorganizados y contienen múltiples problemas de calidad que impiden su análisis directo.

Específicamente, se han identificado los siguientes desafíos:

- Calidad de datos deficiente: Presencia de valores faltantes (5-15% en variables clave), registros duplicados, errores tipográficos en nombres de regiones, y valores atípicos sin validar
- Inconsistencia de formatos: Fechas en múltiples formatos (DD-MM-YYYY, YYYY/MM/DD), unidades no estandarizadas (algunos valores en miles, otros en unidades), categorías escritas de forma inconsistente
- Fragmentación de fuentes: Información distribuida en al menos tres archivos independientes que deben combinarse mediante claves comunes
- Escalas incompatibles: Variables numéricas en rangos muy diferentes que dificultan comparaciones y análisis conjunto
- Falta de validación: No existe un proceso sistemático para verificar la integridad y consistencia de las transformaciones realizadas

Sin un proceso robusto de preprocesamiento y transformación de datos, es imposible generar análisis confiables que sustenten decisiones de política pública educativa.

3. OBJETIVOS DE ESTUDIO

Objetivo General:

Diseñar e implementar un pipeline completo de procesamiento y transformación de datos que consolide información educacional y socioeconómica de Chile, generando un dataset limpio, validado y listo para análisis estadístico.

Objetivos Específicos:

1. Diseñar algoritmos estructurados que resuelvan problemas de limpieza y transformación de datos mediante pseudocódigo y/o diagramas de flujo para al menos dos procesos críticos del pipeline
2. Implementar operaciones de preprocesamiento que incluyan:
 - Detección y tratamiento de valores atípicos (outliers)
 - Manejo de valores faltantes mediante estrategias justificadas
 - Limpieza profunda: estandarización de formatos, eliminación de duplicados, corrección de inconsistencias
3. Aplicar transformaciones de enriquecimiento que comprendan:
 - Normalización de al menos dos variables numéricas con rangos muy diferentes
 - Parseo y estandarización de fechas
 - Codificación de variables categóricas (label encoding o one-hot encoding según corresponda)
 - Creación de al menos una variable derivada (por ejemplo: tasa de matrícula por población)
4. Realizar operaciones de agregación y combinación que involucren:
 - Agregación de datos (estadísticas agrupadas) usando funciones groupby
 - Combinación de al menos dos fuentes de datos mediante operaciones merge/join
 - Consolidación en un dataset final unificado
5. Desarrollar código modular y estructurado siguiendo buenas prácticas:
 - Creación de funciones reutilizables para cada etapa del procesamiento
 - Documentación mediante comentarios y docstrings
 - Nomenclatura clara y consistente
6. Validar el pipeline de transformación mediante:
 - Verificación de totales y conteos antes/después de transformaciones
 - Pruebas de consistencia de los resultados
 - Análisis de eficiencia básica del código

4. METODOLOGÍA SUGERIDA

Se recomienda seguir las siguientes etapas para completar exitosamente el proyecto:

FASE 1: Planificación y Diseño

- Analizar los datasets proporcionados para comprender su estructura
- Identificar los principales problemas de calidad en cada fuente
- Diseñar el flujo general del pipeline (desde datos brutos hasta dataset consolidado)
- Crear pseudocódigo o diagrama de flujo para al menos dos procesos clave (por ejemplo: limpieza de outliers y combinación de datasets)

FASE 2: Preprocesamiento de Datos

- Cargar los datos utilizando pandas
- Explorar cada dataset (shape, info, describe, identificar nulos)
- Implementar funciones para:
 - Detectar y tratar valores atípicos (definir criterio: percentiles, desviación estándar, IQR)
 - Manejar valores faltantes (eliminación, imputación con media/mediana/moda según el caso)
 - Limpiar formatos: estandarizar nombres de regiones, eliminar duplicados, corregir tipos de datos

FASE 3: Transformación y Enriquecimiento

- Normalizar variables numéricas con rangos muy diferentes (usar min-max o estandarización)
- Parsear y estandarizar campos de fecha a formato datetime
- Codificar variables categóricas apropiadamente
- Crear variables derivadas que aporten valor analítico

FASE 4: Agregación y Combinación

- Realizar agregaciones por región u otra variable clave (promedios, sumas, conteos)
- Combinar los datasets limpios mediante merge/join usando claves comunes
- Generar el dataset consolidado final

FASE 5: Validación y Documentación

- Verificar consistencia de totales antes/después de cada transformación
- Documentar decisiones tomadas (por qué se eligió cierta estrategia de imputación, etc.)
- Realizar pruebas básicas de las funciones creadas
- Generar un reporte breve de validación

5. ENTREGABLES Y REQUISITOS ESPECÍFICOS

El/La estudiante deberá entregar un proyecto Python completo que incluya:

A. CÓDIGO FUENTE

Archivo principal: `pipeline_procesamiento.py` o Jupyter Notebook `pipeline_procesamiento.ipynb` que contenga:

1. Sección de Diseño Algorítmico:

- Pseudocódigo o descripción textual detallada del algoritmo para al menos 2 procesos (puede estar en comentarios o en markdown si es notebook)
- Alternativamente, incluir imagen o descripción de diagrama de flujo

2. Funciones Modulares de Preprocesamiento:

- `limpiar_outliers(df, columna, metodo)`: detecta y trata valores atípicos
- `manejar_faltantes(df, estrategia)`: maneja valores nulos con la estrategia especificada
- `estandarizar_formatos(df)`: limpia y uniforma formatos de texto y categorías
- `eliminar_duplicados(df)`: identifica y remueve registros duplicados

3. Funciones de Transformación:

- `normalizar_columna(df, columna, metodo)`: normaliza valores numéricos
- `parsear_fechas(df, columna, formato)`: convierte strings a datetime
- `codificar_categoricas(df, columnas, metodo)`: aplica encoding apropiado
- `crear_variables_derivadas(df)`: genera nuevas columnas calculadas

4. Funciones de Agregación y Combinación:

- `agregar_por_region(df)`: genera estadísticas agrupadas
- `combinar_datasets(df1, df2, clave)`: realiza merge de fuentes

5. Función Principal (Pipeline Completo):

```
python
def ejecutar_pipeline(rutas_archivos):
    """
    Ejecuta el pipeline completo de procesamiento.

    Parámetros:
    rutas_archivos: dict con rutas a los archivos de datos

    Retorna:
    DataFrame consolidado y limpio
    """
    # Orquesta todas las funciones anteriores
    pass
```

6. Validaciones:

- Verificaciones de totales antes/después de transformaciones clave
- Al menos 3 pruebas de consistencia (asserts o prints con verificaciones)
- Manejo básico de errores con try/except en operaciones críticas

Requisitos técnicos del código:

- Todas las funciones deben tener docstrings explicando propósito, parámetros y retorno
- Comentarios en secciones complejas del código
- Nomenclatura clara para variables y funciones (snake_case)
- Modularidad: evitar bloques de código repetido
- Uso apropiado de estructuras de control (if/else, bucles)
- Preferir operaciones vectorizadas de pandas cuando sea posible

B. INFORME DE PROCESO (20% de la evaluación)

Archivo: informe_proceso.pdf o informe_proceso.md (2-4 páginas) que incluya:

1. Descripción de datasets originales:

- Fuentes utilizadas y estructura inicial
- Principales problemas identificados en cada uno

2. Decisiones de preprocesamiento:

- Justificación de estrategias para outliers (¿por qué ese umbral?)
- Justificación de imputación de faltantes (¿por qué media, mediana o eliminación?)
- Explicación de limpieza de formatos realizada

3. Transformaciones aplicadas:

- Qué variables se normalizaron y por qué
- Qué codificación se usó para categóricas y por qué
- Qué variables derivadas se crearon y su utilidad

4. Proceso de validación:

- Resumen de verificaciones realizadas
- Comparación de totales antes/después
- Problemas encontrados y cómo se resolvieron

5. Reflexión sobre eficiencia:

- Breve análisis de complejidad de las operaciones principales
- Identificación de posibles cuellos de botella

C. DATASET FINAL CONSOLIDADO

Archivo: datos_consolidados.csv

- Dataset resultante del pipeline, limpio y listo para análisis
- Debe incluir todas las transformaciones aplicadas
- Formato consistente en todas las columnas
- Sin valores faltantes no justificados
- Sin duplicados

DATASETS PROPORCIONADOS (Simulación - Descripción)

Se proporcionarán 3 archivos:

1. rendimiento_escolar.csv: Datos de establecimientos educacionales
 - Columnas: codigo_establecimiento, nombre, region, promedio_simce, matricula, fecha_medicion, tipo_establecimiento
 - Problemas: fechas en formato inconsistente, valores atípicos en promedio_simce, algunos duplicados, nombres de región con mayúsculas/minúsculas mixtas
2. indicadores_regionales.json: Datos socioeconómicos por región
 - Campos: codigo_region, nombre_region, poblacion, ingreso_promedio, indice_vulnerabilidad
 - Problemas: algunos valores faltantes en ingreso_promedio, nombres de región no coinciden exactamente con el CSV
3. infraestructura_educativa.csv: Recursos educativos
 - Columnas: codigo_region, num_establecimientos, presupuesto_educacion, año
 - Problemas: presupuesto en diferentes unidades (miles/millones sin estandarizar), algunos registros con año faltante

RESUMEN EJECUTIVO DE LA EVALUACIÓN

El/La estudiante debe:

1. Diseñar el flujo de procesamiento mediante pseudocódigo o diagrama de flujo
2. Implementar un pipeline completo en Python con funciones modulares que:
 - Limpie datos (outliers, faltantes, duplicados, formatos)
 - Transformen datos (normalización, parseo fechas, encoding, variables derivadas)
 - Agreguen y combinen múltiples fuentes de datos
3. Validar que las transformaciones son correctas mediante verificaciones
4. Documentar el proceso y decisiones en un informe breve
5. Entregar el código funcional, el informe y el dataset consolidado final

Formato de entrega: Carpeta comprimida (.zip) conteniendo todos los archivos mencionados